MOVIEMIND

SENTIMENT ANALYSIS ON MOVIE REVIEWS USING MACHINE LEARNING AND NATURAL LANGUAGE PROCESSING

ABSTRACT

Sentiment analysis is an important application of Natural Language Processing (NLP) that identifies opinions and emotions expressed in textual data. With the increasing availability of online movie reviews, analyzing audience opinions manually has become difficult and time-consuming. This project, titled **"Sentiment Analysis on Movie Reviews Using Machine Learning and NLP,"** presents an automated approach for classifying movie reviews into positive and negative sentiments.

The proposed system uses the NLTK Movie Reviews dataset, which contains 2,000 movie reviews equally divided into positive and negative categories. The collected textual data is cleaned and preprocessed using techniques such as lowercasing, removal of HTML tags and URLs, tokenization, stopword removal, and lemmatization. The preprocessed reviews are converted into numerical feature vectors using Term Frequency–Inverse Document Frequency (TF-IDF).

Three machine learning algorithms are implemented and evaluated: Multinomial Naive Bayes, Linear Support Vector Machine (SVM), and Logistic Regression. The dataset is divided into training and testing sets using an 80:20 ratio. Their performance is evaluated using accuracy, precision, recall, F1-score, and confusion matrices.

Among the implemented algorithms, Linear SVM achieved the highest test accuracy of **85.25%**, with an F1-score of **85.50%**. The trained SVM model is integrated into a Flask-based web application called **MovieMind**, which allows users to enter movie reviews and receive real-time sentiment predictions. An interactive dashboard presents model evaluation results and prediction history.

The project demonstrates the practical application of NLP and machine learning in opinion mining and automated review classification.

Keywords: Sentiment Analysis, Natural Language Processing, Machine Learning, Movie Reviews, TF-IDF, Support Vector Machine, Flask, Text Classification.

CHAPTER 1: INTRODUCTION

1.1 Introduction

The rapid growth of digital platforms has made online reviews an important source of information for understanding customer opinions and public feedback. Movie viewers frequently share their experiences and opinions through online review websites, discussion forums, and social media platforms. These reviews contain valuable information about movie quality, acting, storytelling, direction, and overall audience satisfaction.

However, manually analyzing a large number of reviews is time-consuming and inefficient. Sentiment analysis provides an automated solution by identifying the emotional orientation of textual information. It is commonly used to classify opinions as positive, negative, or neutral.

Sentiment analysis is a subfield of Natural Language Processing (NLP), which enables computers to process, understand, and analyze human language. Machine learning algorithms can learn patterns from previously labeled text data and use those patterns to classify new, unseen reviews.

This project develops a machine-learning-based sentiment analysis system that classifies movie reviews into positive and negative categories. It combines text preprocessing, feature extraction, classification algorithms, model evaluation, and web application development.

1.2 Problem Statement

Online movie platforms contain thousands of user-generated reviews expressing different opinions. Extracting the overall sentiment from these reviews manually requires significant time and effort.

Traditional manual analysis may also introduce inconsistency and human bias. Therefore, an automated system is required to process textual reviews and classify them accurately into positive and negative sentiments.

The main problem addressed by this project is to develop an efficient sentiment classification system using Natural Language Processing and machine learning techniques and provide an easy-to-use interface for real-time predictions.

1.3 Objectives

The major objectives of the project are:

1. To collect and analyze a labeled movie review dataset.
2. To perform text preprocessing using NLP techniques.
3. To convert textual reviews into numerical representations using TF-IDF.
4. To implement multiple machine learning classification algorithms.
5. To compare model performance using standard evaluation metrics.
6. To identify the best-performing classification algorithm.
7. To develop a Flask-based web application for real-time sentiment prediction.
8. To visualize model performance and prediction history through a dashboard.

1.4 Scope of the Project

The scope of this project focuses on binary sentiment classification of movie reviews.

The system accepts an English-language movie review as input and predicts whether the expressed opinion is positive or negative.

The project includes:

- Processing and cleaning movie review text.
- Extracting textual features using TF-IDF.
- Training and evaluating three machine learning algorithms.

- Saving trained models for future predictions.
- Integrating the selected model into a Flask web application.
- Displaying evaluation metrics and confusion matrices.
- Maintaining a record of submitted predictions.

The current implementation is limited to English-language reviews and binary sentiment classification. It does not perform aspect-based sentiment analysis or multilingual sentiment detection.

1.5 Motivation

Online reviews strongly influence audience decisions and movie popularity. Understanding public opinion can help viewers, filmmakers, and entertainment platforms make informed decisions.

The motivation behind this project is to demonstrate how machine learning can automate the process of understanding textual opinions. It also provides practical experience in NLP preprocessing, feature engineering, classification, model evaluation, and full-stack application development.

1.6 Organization of the Report

CHAPTER 2: LITERATURE REVIEW

2.1 Overview

Sentiment analysis has become an important research area in Natural Language Processing (NLP). It focuses on identifying opinions, emotions, and subjective information expressed in text. Researchers have developed several approaches to automatically classify reviews and opinions using rule-based methods, machine learning algorithms, and deep learning techniques.

Movie reviews are a common benchmark for sentiment classification because they contain subjective expressions about acting, direction, screenplay, music, and overall movie quality.

This chapter discusses the fundamental concepts, commonly used techniques, and research contributions related to movie review sentiment analysis.

2.2 Existing Approaches to Sentiment Analysis

Sentiment analysis techniques can broadly be classified into the following categories.

2.2.1 Lexicon-Based Approach

The lexicon-based approach uses a predefined dictionary containing words associated with positive and negative sentiment.

For example:

- Positive words: excellent, wonderful, amazing, enjoyable.
- Negative words: terrible, boring, disappointing, poor.

The sentiment of a review is determined by analyzing the sentiment scores of the words it contains.

Advantages:

- Simple to implement.
- Does not require a labeled training dataset.
- Easy to understand and interpret.

Limitations:

- Difficult to understand context and sarcasm.
- Performance depends on the quality of the sentiment dictionary.
- May incorrectly classify sentences containing negation.

2.2.2 Machine Learning-Based Approach

Machine learning approaches use labeled training data to learn patterns associated with different sentiment categories.

The text is first converted into numerical features using techniques such as Bag of Words or TF-IDF. Classification algorithms are then trained using these features.

Common machine learning algorithms include:

- Naive Bayes
- Support Vector Machine
- Logistic Regression
- Decision Tree
- Random Forest

Advantages:

- Learns patterns from labeled examples.
- Can be evaluated using standard performance metrics.
- Suitable for automated classification of large collections of reviews.

Limitations:

- Requires labeled training data.
- Performance depends on feature extraction and data quality.
- May struggle with complex language and context.

2.2.3 Deep Learning-Based Approach

Deep learning methods use neural networks to learn complex patterns from textual data. Architectures such as Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Transformer models can capture relationships between words and their surrounding context.

These methods can achieve strong results when sufficient training data and computational resources are available.

Advantages:

- Can learn complex textual patterns.

- Can capture contextual relationships.
- Supports advanced language understanding.

Limitations:

- Often requires larger datasets.
- Requires more computational resources.
- Training and tuning can be more complex.

2.3 Important Research Contributions

The following research works provide a foundation for sentiment analysis and text classification.

2.3.1 Pang, Lee, and Vaithyanathan (2002)

In their research paper, *Thumbs up? Sentiment Classification Using Machine Learning Techniques*, Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan investigated the use of machine learning algorithms for classifying movie reviews.

The study explored algorithms such as Naive Bayes, Maximum Entropy, and Support Vector Machines. It demonstrated that traditional machine learning methods can be applied effectively to movie review sentiment classification.

Relevance to this project: This research provides an important foundation for comparing machine learning classification algorithms on movie review data.

2.3.2 Pang and Lee (2004)

In *A Sentimental Education: Sentiment Analysis Using Subjectivity Summarization Based on Minimum Cuts*, Pang and Lee explored techniques for identifying subjective portions of text before performing sentiment analysis.

The work highlighted the importance of distinguishing subjective expressions from objective information.

Relevance to this project: It demonstrates the importance of analyzing textual content and identifying opinion-bearing expressions.

2.3.3 Joachims (1998)

Thorsten Joachims presented *Text Categorization with Support Vector Machines: Learning with Many Relevant Features*.

The work investigated the effectiveness of Support Vector Machines for text categorization, particularly when documents contain large numbers of features.

Relevance to this project: It supports the use of SVM for high-dimensional text classification tasks such as TF-IDF-based movie review analysis.

2.3.4 Salton and Buckley (1988)

In *Term-Weighting Approaches in Automatic Text Retrieval*, Gerard Salton and Christopher Buckley discussed different term-weighting methods for representing documents.

The research contributed to the development and understanding of term weighting techniques used in information retrieval and text processing.

Relevance to this project: It provides background for representing movie reviews numerically using term-weighting techniques such as TF-IDF.

2.4 Comparison of Existing Approaches

Approach	Main Technique	Advantages	Limitations
Lexicon-Based	Sentiment dictionaries	Simple and interpretable	Limited contextual understanding
Machine Learning	Statistical classification	Effective with labeled data	Depends on feature quality
Deep Learning	Neural networks	Learns complex patterns	Requires more resources

2.5 Research Gap

Existing sentiment analysis approaches demonstrate that machine learning can effectively classify textual opinions. However, several practical challenges remain:

1. Textual reviews contain noise, spelling variations, and irrelevant information.
2. Word importance varies across different reviews.
3. Negation can change the sentiment of a sentence.
4. Different classification algorithms may produce different results.
5. Model performance needs to be presented in an understandable manner.
6. A trained classification model should be integrated into an accessible application.

The proposed MovieMind system addresses these practical requirements by combining text preprocessing, TF-IDF feature extraction, comparative evaluation of three machine learning algorithms, and a Flask-based web interface.

2.6 Proposed Approach

Based on the reviewed approaches, this project adopts a supervised machine learning methodology.

The proposed system performs the following operations:

1. Collect labeled movie reviews from the NLTK Movie Reviews corpus.
2. Clean and preprocess the review text.
3. Extract numerical features using TF-IDF.
4. Train Naive Bayes, Linear SVM, and Logistic Regression classifiers.
5. Evaluate the classifiers using accuracy, precision, recall, F1-score, and confusion matrices.
6. Select the best-performing model based on experimental results.
7. Integrate the selected model into the MovieMind web application.

The experimental results show that Linear SVM achieved the highest accuracy of 85.25% among the three implemented classifiers.

2.7 Chapter Summary

This chapter reviewed major approaches used in sentiment analysis, including lexicon-based, traditional machine learning, and deep learning techniques. It also discussed important research contributions related to movie review classification, text categorization, and term weighting.

CHAPTER 3: METHODOLOGY

3.1 Introduction

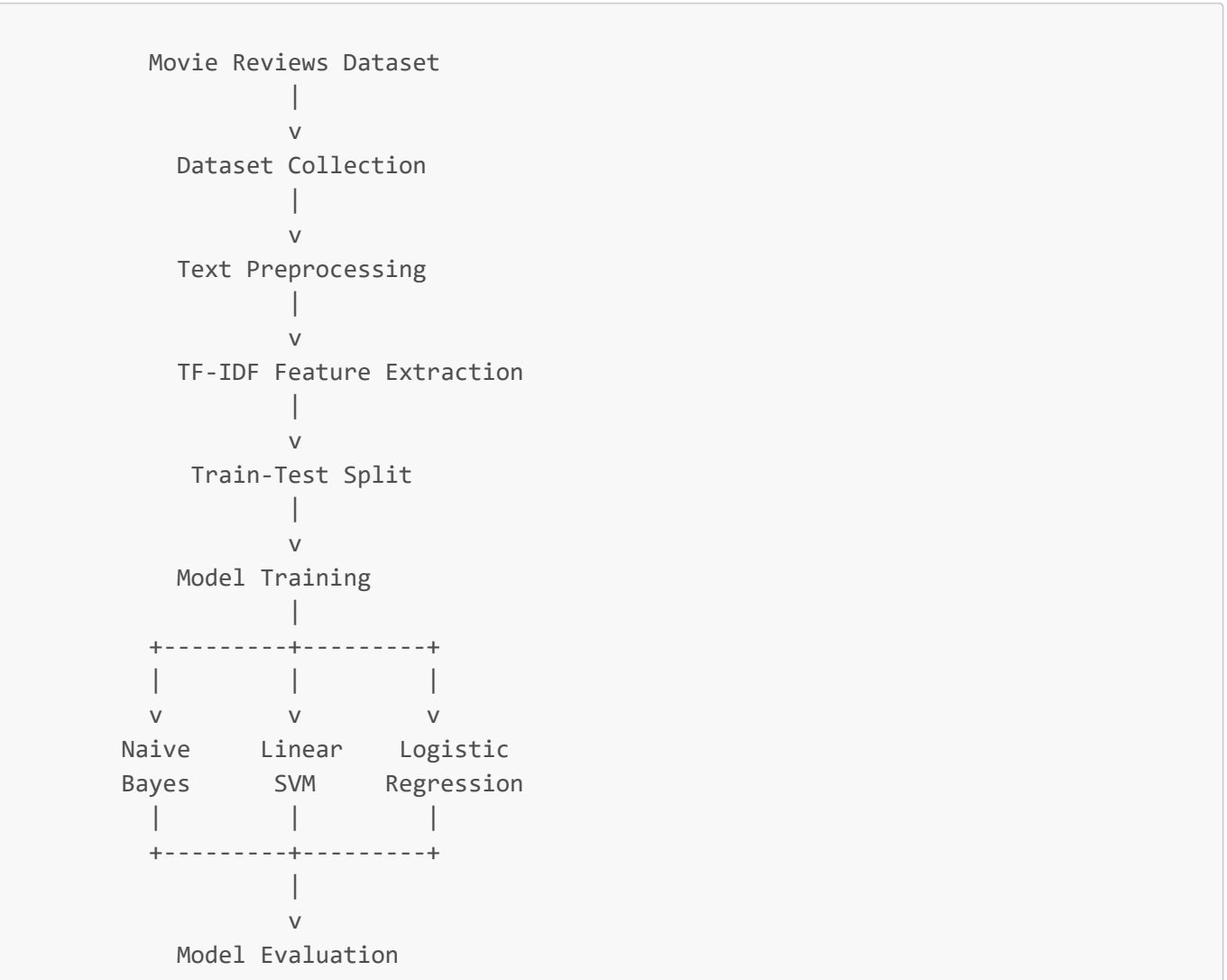
The methodology describes the systematic approach followed to develop the MovieMind sentiment analysis system. The project uses supervised machine learning techniques combined with Natural Language Processing to classify movie reviews into positive and negative categories.

The implementation consists of dataset collection, text preprocessing, feature extraction, dataset splitting, model training, model evaluation, and web application integration.

3.2 Proposed System Architecture

The proposed system follows a sequential machine learning workflow.

System workflow:



```
                |
                v
        Best Model Selection
                |
                v
        Flask Web Application
                |
                v
        Sentiment Prediction
                |
                v
        Dashboard and Results
```

The system first collects the labeled movie reviews and preprocesses the text. TF-IDF converts the reviews into numerical feature vectors. Three classification algorithms are trained and evaluated, and the best-performing model is integrated into the web application.

3.3 Dataset Collection

The project uses the Movie Reviews corpus provided by the Natural Language Toolkit (NLTK).

The corpus contains movie reviews collected for research on sentiment classification.

Dataset characteristics:

Attribute	Description
Dataset Name	NLTK Movie Reviews
Total Reviews	2,000
Positive Reviews	1,000
Negative Reviews	1,000
Data Type	Text
Classification Type	Binary Classification
Input	Movie Review
Output	Positive or Negative

The dataset is balanced, with an equal number of positive and negative reviews.

The corpus is converted into a CSV file named `movie_reviews.csv`, containing two columns:

- `review`: The original movie review text.
- `sentiment`: The corresponding sentiment label.

3.4 Data Preprocessing

Raw textual data may contain HTML tags, URLs, special characters, unnecessary words, and different forms of the same word. These elements can affect feature extraction and model performance.

Therefore, preprocessing is performed to convert the reviews into a consistent textual format.

The following steps are implemented.

3.4.1 Lowercasing

All text is converted into lowercase to ensure that words with different capitalization are treated consistently.

Example:

Input: `THIS MOVIE WAS EXCELLENT`

Output: `this movie was excellent`

3.4.2 HTML Tag Removal

HTML tags are removed because they do not contribute meaningful sentiment information.

Example:

Input: `<p>This movie is excellent</p>`

Output: `This movie is excellent`

3.4.3 URL Removal

Web addresses are removed from the review text to eliminate irrelevant links.

Example:

Input: `Amazing movie! Visit https://example.com`

Output: `Amazing movie! Visit`

3.4.4 Special Character Removal

Unwanted punctuation, digits, and special characters are removed to retain alphabetic text.

Example:

Input: `The movie was amazing!!! 100% worth watching.`

Output: `The movie was amazing worth watching`

3.4.5 Tokenization

Tokenization divides a sentence into individual words called tokens.

Example:

Input: `the movie was very interesting`

Output: `['the', 'movie', 'was', 'very', 'interesting']`

3.4.6 Stopword Removal

Stopwords are common words that may contribute less to classification in many text-processing tasks.

Examples include:

- the
- is
- was
- at
- and

These words are removed during preprocessing. However, important negation words such as `not`, `no`, and `nor` are preserved because they can change the meaning of a review.

For example:

- The movie was good.
- The movie was not good.

Preserving negation helps maintain this important sentiment difference.

3.4.7 Lemmatization

Lemmatization converts words into their base dictionary forms.

Examples:

Original Word	Lemmatized Word
movies	movie
acting	acting
enjoyed	enjoy
better	better
characters	character

The project uses the WordNet Lemmatizer from NLTK.

After preprocessing, the cleaned reviews are saved in `processed_movie_reviews.csv`.

3.5 Feature Extraction Using TF-IDF

Machine learning algorithms require numerical input. Therefore, the cleaned textual reviews are converted into numerical feature vectors using Term Frequency–Inverse Document Frequency (TF-IDF).

TF-IDF assigns importance to a word based on how frequently it occurs in a document and how rare it is across the complete collection of documents.

3.5.1 Term Frequency (TF)

Term Frequency measures how often a term occurs in a document.

$$TF(t,d) = \frac{\text{Count of term } t \text{ in document } d}{\text{Total terms in document } d}$$

3.5.2 Inverse Document Frequency (IDF)

Inverse Document Frequency measures how important a term is across the document collection.

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right)$$

Where:

- N = Total number of documents.
- $DF(t)$ = Number of documents containing term t.

3.5.3 TF-IDF Calculation

The TF-IDF score is calculated as:

$$TFIDF(t,d) = TF(t,d) \times IDF(t)$$

Words that occur frequently in a particular review but less frequently across the entire collection may receive higher weights.

3.5.4 TF-IDF Configuration

The following configuration is used in this project:

Parameter	Value
Vectorizer	TfidfVectorizer
Maximum Features	5,000
N-gram Range	(1, 2)
Minimum N-gram	Unigram
Maximum N-gram	Bigram

Unigrams represent individual words, while bigrams represent consecutive pairs of words.

For example:

Text: `the movie was excellent`

Unigrams: `the`, `movie`, `was`, `excellent`

Bigrams: `the movie`, `movie was`, `was excellent`

Using both unigrams and bigrams helps the model capture individual words and some short word combinations.

3.6 Train-Test Split

The dataset is divided into training and testing subsets to evaluate how well the trained model performs on unseen data.

The project uses an 80:20 split.

Dataset Portion	Percentage	Number of Reviews
Training Set	80%	1,600
Testing Set	20%	400
Total	100%	2,000

A stratified split is used to preserve the positive and negative class proportions in both subsets.

The random state is set to 42 to make the split reproducible.

The TF-IDF vectorizer is fitted only on the training data and then used to transform the testing data. This avoids using information from the test set during feature learning.

3.7 Machine Learning Algorithms

Three supervised machine learning algorithms are implemented and compared.

3.7.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It estimates the probability that a review belongs to a particular sentiment category using the features extracted from the text.

Bayes' theorem is represented as:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Where:

- $P(C|X)$ = Probability of class C given features X.
- $P(X|C)$ = Probability of features given class C.
- $P(C)$ = Prior probability of the class.
- $P(X)$ = Probability of the features.

It is commonly used for text classification because of its computational efficiency.

3.7.2 Support Vector Machine (SVM)

Support Vector Machine is a supervised learning algorithm that identifies an optimal decision boundary separating different classes.

In this project, LinearSVC is used to classify positive and negative movie reviews.

The algorithm attempts to maximize the margin between the two sentiment classes. It is suitable for high-dimensional text representations such as TF-IDF features.

3.7.3 Logistic Regression

Logistic Regression is a supervised classification algorithm that estimates the probability of a sample belonging to a particular class.

The logistic function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The model uses the estimated probability to assign a sentiment category.

Logistic Regression is widely used in binary classification because it is relatively efficient and provides a probabilistic interpretation of its predictions.

3.8 Model Evaluation Metrics

The performance of each classification algorithm is evaluated using the following metrics.

3.8.1 Accuracy

Accuracy measures the proportion of correctly classified reviews.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

3.8.2 Precision

Precision measures the proportion of predicted positive reviews that are actually positive.

$$Precision = \frac{TP}{TP + FP}$$

3.8.3 Recall

Recall measures the proportion of actual positive reviews correctly identified.

$$Recall = \frac{TP}{TP + FN}$$

3.8.4 F1-Score

F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.8.5 Confusion Matrix

A confusion matrix provides a detailed comparison between actual and predicted sentiment categories.

It consists of:

- True Positive (TP)
- True Negative (TN)
- False Positive (FP)
- False Negative (FN)

These values help identify the types of classification errors made by the model.

3.9 Best Model Selection

After training and evaluating the three algorithms, their performance metrics are compared.

The experimental results are:

Algorithm	Accuracy
Multinomial Naive Bayes	81.25%
Linear SVM	85.25%
Logistic Regression	83.50%

Linear SVM achieved the highest accuracy of 85.25%. Therefore, it is selected as the final prediction model for the MovieMind application.

The trained model and TF-IDF vectorizer are saved using Joblib so that they can be loaded later without retraining.

3.10 Chapter Summary

This chapter explained the methodology used to develop the MovieMind sentiment analysis system. It described the dataset, text preprocessing techniques, TF-IDF feature extraction, train-test split, classification algorithms, and evaluation metrics.

The next chapter presents the system design and implementation, including the Flask web application, user interface, prediction process, and analytics dashboard.

CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION

4.1 Introduction

This chapter describes the system architecture, software requirements, modules, and implementation details of the MovieMind sentiment analysis application.

The proposed system combines machine learning techniques with a web-based interface. The trained Linear Support Vector Machine model is integrated with Flask to provide real-time sentiment predictions.

The application also includes an analytics dashboard that displays model performance metrics, confusion matrices, and prediction history.

4.2 System Architecture

MovieMind follows a modular architecture consisting of a data processing layer, machine learning layer, application layer, and presentation layer.

4.2.1 Architecture Components

1. Data Layer

The data layer contains the original movie review dataset and the preprocessed dataset. It stores review text and corresponding sentiment labels.

2. NLP Processing Layer

This layer performs text cleaning, tokenization, stopword removal, and lemmatization.

3. Feature Extraction Layer

The TF-IDF vectorizer converts preprocessed text into numerical feature vectors.

4. Machine Learning Layer

Three classification algorithms are trained and evaluated. The best-performing model, Linear SVM, is selected for final predictions.

5. Application Layer

Flask manages user requests, loads the trained model, processes review input, and returns sentiment predictions.

6. Presentation Layer

The frontend displays the MovieMind home page, prediction results, and analytics dashboard.

4.2.2 Architecture Diagram



```
+----------------------------------+
|        TRAINED SVM MODEL         |
|       Sentiment Classification   |
+----------------+-----------------+
                 |
                 v
+----------------------------------+
|       PREDICTION RESULT          |
|       Positive / Negative        |
+----------------+-----------------+
                 |
                 v
+----------------------------------+
|       WEB INTERFACE              |
|       MovieMind Dashboard        |
+----------------------------------+
```

4.3 Hardware and Software Requirements

4.3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	2 GB available space
Operating System	Windows 10 or above
Internet	Required for initial package and NLTK downloads

These are suggested minimum development requirements, not measured performance limits.

4.3.2 Software Requirements

Software	Purpose
Python	Main programming language
Visual Studio Code	Development environment
Flask	Web application framework
NLTK	Natural Language Processing
Pandas	Dataset processing
NumPy	Numerical operations
Scikit-learn	Machine learning algorithms
Matplotlib	Data visualization

Software	Purpose
Seaborn	Confusion matrix visualization
Joblib	Saving and loading trained models
HTML	Web page structure
CSS	Web page styling
JavaScript	Interactive dashboard components
Chart.js	Dashboard charts

4.4 Project Modules

The project is divided into the following modules.

4.4.1 Dataset Collection Module

File: `collect_dataset.py`

This module downloads and collects movie reviews from the NLTK Movie Reviews corpus.

It extracts the review text and sentiment labels and stores them in a CSV file.

Output: `dataset/movie_reviews.csv`

4.4.2 Dataset Exploration Module

File: `explore_dataset.py`

This module analyzes the collected dataset and generates visualizations.

Its functions include:

- Displaying dataset dimensions.
- Checking positive and negative review counts.
- Analyzing review lengths.
- Generating sentiment distribution charts.
- Visualizing review length distribution.

Output visualizations are stored in the `results/` directory.

4.4.3 Text Preprocessing Module

File: `preprocessing.py`

This module prepares the textual reviews for machine learning.

It performs:

- Lowercasing.
- HTML and URL removal.

- Special character removal.
- Tokenization.
- Stopword removal.
- Lemmatization.

The processed reviews are saved in:

`dataset/processed_movie_reviews.csv`

4.4.4 Feature Extraction Module

File: `feature_extraction.py`

This module converts the processed movie reviews into numerical feature vectors using TF-IDF.

It also divides the dataset into training and testing sets using an 80:20 stratified split.

The vectorizer and prepared feature data are stored in the `models/` directory.

4.4.5 Model Training Module

File: `train_model.py`

This module trains and evaluates three machine learning algorithms:

1. Multinomial Naive Bayes.
2. Linear Support Vector Machine.
3. Logistic Regression.

It calculates accuracy, precision, recall, and F1-score and generates confusion matrices and comparison charts.

The trained models are saved using Joblib.

4.4.6 Prediction Testing Module

File: `test_predictions.py`

This module loads the trained SVM model and TF-IDF vectorizer to test sentiment predictions on sample reviews.

It verifies whether the model can classify positive and negative movie reviews.

4.4.7 Web Application Module

File: `app.py`

This module connects the trained machine learning model with the Flask web application.

Its main responsibilities include:

- Loading the trained SVM model.
- Loading the TF-IDF vectorizer.
- Preprocessing user-submitted reviews.

- Converting reviews into TF-IDF features.
- Predicting positive or negative sentiment.
- Saving prediction history.
- Displaying dashboard information.

4.4.8 Frontend Module

The frontend is developed using HTML, CSS, and JavaScript.

The main frontend files are:

- `templates/index.html`
- `templates/dashboard.html`
- `static/css/style.css`

The home page accepts movie reviews and displays sentiment predictions. The dashboard presents model evaluation results, comparison charts, confusion matrices, and prediction history.

4.5 Implementation Details

4.5.1 Model Loading

The trained SVM model and TF-IDF vectorizer are loaded using Joblib.

```
import joblib

model = joblib.load("models/svm_model.pkl")

vectorizer = joblib.load(
    "models/tfidf_vectorizer.pkl"
)
```

This allows the application to use previously trained models without repeating the training process every time the server starts.

4.5.2 Review Preprocessing

When a user submits a review, the application performs the same preprocessing operations used during model development.

The cleaned review is then passed to the TF-IDF vectorizer.

Consistency between training-time and prediction-time preprocessing is important for reliable classification.

4.5.3 Feature Transformation

The processed review is converted into a numerical vector.

```
review_vector = vectorizer.transform(
    [processed_review]
)
```

The resulting vector is used as input to the trained SVM classifier.

4.5.4 Sentiment Prediction

The trained model predicts the sentiment category.

```
prediction = model.predict(review_vector)

if prediction[0] == "pos":
    sentiment = "Positive"
else:
    sentiment = "Negative"
```

The application displays the predicted sentiment to the user.

4.5.5 Prediction History

The application stores submitted reviews and their predicted sentiment in a CSV file.

Output:

`results/prediction_history.csv`

This allows the dashboard to display previously submitted predictions.

4.6 User Interface Design

MovieMind provides a simple and responsive web interface.

4.6.1 Home Page

The home page provides:

- MovieMind branding.
- Navigation links.
- Review input area.
- Sentiment prediction button.
- Positive or negative result display.
- Project information and feature descriptions.

Screenshot Placeholder – Figure 4.1: MovieMind Home Page

[Insert the actual screenshot of the MovieMind home page here.]

4.6.2 Dashboard Page

The dashboard provides an overview of model performance and prediction activity.

Its main components include:

- Total model count.
- Best-performing algorithm.
- Best model accuracy.
- Positive and negative prediction counts.
- Model accuracy comparison chart.
- Precision, recall, and F1-score comparison chart.
- Model performance table.
- Confusion matrix visualizations.
- Recent prediction history.

Screenshot Placeholder – Figure 4.2: MovieMind Analytics Dashboard

[Insert the actual screenshot of the dashboard here.]

4.7 Application Execution Workflow

The application follows these steps when a user submits a movie review:

1. The user opens the MovieMind home page.
2. The user enters a movie review in the text area.
3. The review is submitted to the Flask server.
4. The server preprocesses the input text.
5. The TF-IDF vectorizer transforms the review into numerical features.
6. The trained SVM model predicts the sentiment.
7. The predicted category is displayed on the home page.
8. The prediction is saved in the history CSV file.
9. The dashboard retrieves the available prediction history and displays it.

4.8 Advantages of the Proposed System

- Automates the classification of movie reviews.
- Uses multiple machine learning algorithms for comparison.
- Applies NLP preprocessing to improve text consistency.
- Uses TF-IDF to represent textual information numerically.
- Provides real-time predictions through a web interface.
- Displays model performance through visualizations.
- Stores prediction history for reference.
- Uses saved models to avoid retraining for every prediction.

4.9 Chapter Summary

This chapter explained the system architecture, hardware and software requirements, individual project modules, and implementation details of MovieMind.

It also described how Flask integrates the trained SVM model with the frontend to provide real-time sentiment predictions. The next chapter presents the experimental results, performance evaluation, and discussion of the

implemented machine learning algorithms.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Introduction

This chapter presents the experimental results obtained from the MovieMind sentiment analysis system. The performance of three machine learning algorithms is evaluated using the test dataset.

The implemented algorithms are Multinomial Naive Bayes, Linear Support Vector Machine (SVM), and Logistic Regression.

The evaluation is performed using accuracy, precision, recall, F1-score, and confusion matrices. The results are compared to identify the most suitable algorithm for movie review sentiment classification.

5.2 Experimental Setup

The experiments were conducted using Python and the Scikit-learn machine learning library.

The NLTK Movie Reviews dataset contains 2,000 reviews, divided equally into positive and negative categories.

The dataset was divided into training and testing sets using an 80:20 stratified split.

Parameter	Configuration
Total Reviews	2,000
Training Samples	1,600
Testing Samples	400
Positive Reviews	1,000
Negative Reviews	1,000
Feature Extraction	TF-IDF
Maximum Features	5,000
N-gram Range	(1, 2)
Random State	42
Evaluation Method	Classification Metrics

5.3 Model Performance Comparison

Three machine learning algorithms were trained and evaluated using the same train-test split.

The following table presents the experimental results.

Algorithm	Accuracy	Precision	Recall	F1-Score
Multinomial Naive Bayes	81.25%	82.05%	80.00%	81.01%

Algorithm	Accuracy	Precision	Recall	F1-Score
Linear SVM	85.25%	84.06%	87.00%	85.50%
Logistic Regression	83.50%	82.52%	85.00%	83.74%

The results indicate that Linear SVM achieved the highest accuracy and F1-score among the three implemented algorithms.

5.4 Accuracy Comparison

Accuracy represents the percentage of correctly classified reviews among all testing samples.

The obtained accuracy values are:

- Multinomial Naive Bayes: 81.25%.
- Linear SVM: 85.25%.
- Logistic Regression: 83.50%.

Linear SVM achieved the highest accuracy of 85.25%, outperforming Naive Bayes by 4 percentage points and Logistic Regression by 1.75 percentage points.

Figure 5.1: Model Accuracy Comparison

[Insert `results/model_accuracy_comparison.png` here.]

5.5 Precision, Recall, and F1-Score Analysis

Precision, recall, and F1-score provide additional information about the classification performance.

5.5.1 Precision

Linear SVM achieved a precision of 84.06%, indicating that 84.06% of the reviews predicted as positive were actually positive under the evaluation convention used.

5.5.2 Recall

Linear SVM achieved a recall of 87.00%, indicating that it correctly identified 87.00% of the actual positive reviews.

5.5.3 F1-Score

Linear SVM achieved an F1-score of 85.50%, demonstrating a balanced relationship between precision and recall.

Figure 5.2: Model Performance Comparison

[Insert `results/model_performance_comparison.png` here.]

5.6 Confusion Matrix Analysis

A confusion matrix provides a detailed view of the correct and incorrect predictions made by a classification model.

The project generates confusion matrices for all three algorithms.

5.6.1 Naive Bayes Confusion Matrix

Figure 5.3: Naive Bayes Confusion Matrix

[Insert `results/naive_bayes_confusion_matrix.png` here.]

The confusion matrix illustrates the correct positive and negative predictions as well as the misclassified reviews.

5.6.2 Linear SVM Confusion Matrix

Figure 5.4: Linear SVM Confusion Matrix

[Insert `results/svm_confusion_matrix.png` here.]

The Linear SVM model achieved the highest overall accuracy among the implemented algorithms. Its confusion matrix helps examine the distribution of correct and incorrect classifications.

5.6.3 Logistic Regression Confusion Matrix

Figure 5.5: Logistic Regression Confusion Matrix

[Insert `results/logistic_regression_confusion_matrix.png` here.]

The Logistic Regression model achieved 83.50% accuracy. Its confusion matrix provides additional insight into its classification performance.

5.7 Sample Sentiment Predictions

The trained SVM model was tested using sample movie reviews to verify its prediction functionality.

The following examples were successfully classified during testing.

Input Review	Actual Sentiment	Predicted Sentiment
Fantastic and amazing movie	Positive	Positive
The movie was boring	Negative	Negative
Excellent acting and wonderful story	Positive	Positive
Terrible movie with poor direction	Negative	Negative

The sample testing demonstrates that the application can process new review text and return a sentiment prediction.

Figure 5.6: Positive Sentiment Prediction

[Insert an actual screenshot of a positive prediction here.]

Figure 5.7: Negative Sentiment Prediction

[Insert an actual screenshot of a negative prediction here.]

5.8 Web Application Results

The trained SVM model was integrated into the Flask-based MovieMind web application.

The application provides the following functionality:

1. Accepts movie reviews from users.
2. Preprocesses the submitted text.
3. Converts the review into TF-IDF features.
4. Predicts positive or negative sentiment.
5. Displays the result on the web page.
6. Stores prediction history.
7. Displays model evaluation results through the dashboard.

Figure 5.8: MovieMind Home Page

[Insert the actual home page screenshot here.]

Figure 5.9: MovieMind Analytics Dashboard

[Insert the actual dashboard screenshot here.]

5.9 Discussion

The experimental results demonstrate that all three implemented algorithms can classify movie reviews with reasonable performance.

Among the evaluated algorithms, Linear SVM achieved the highest accuracy of 85.25%, followed by Logistic Regression at 83.50% and Multinomial Naive Bayes at 81.25%.

The strong performance of Linear SVM can be attributed to its ability to identify effective decision boundaries in high-dimensional feature spaces. This characteristic makes it suitable for classification tasks using TF-IDF representations.

Logistic Regression also achieved competitive results, while Multinomial Naive Bayes provided a relatively simple and computationally efficient baseline.

The comparison highlights the importance of selecting an appropriate classification algorithm for textual data.

The MovieMind web application demonstrates how the trained model can be integrated into a practical system for real-time sentiment prediction.

5.10 Limitations

Although the proposed system provides useful results, some limitations remain:

1. The dataset contains only 2,000 reviews.
2. The system supports only positive and negative sentiment categories.
3. The current implementation focuses on English-language reviews.
4. Sarcasm and complex contextual expressions may be difficult to classify.
5. The model uses traditional machine learning rather than contextual deep learning models.
6. Performance on movie reviews from other sources may differ from the reported test results.

5.11 Chapter Summary

This chapter presented the experimental evaluation of the MovieMind sentiment analysis system.

Three machine learning algorithms were compared using accuracy, precision, recall, F1-score, and confusion matrices. Linear SVM achieved the best performance, with 85.25% accuracy and an 85.50% F1-score.

The results demonstrate the usefulness of combining NLP preprocessing, TF-IDF feature extraction, and supervised machine learning for automated movie review sentiment classification.

CHAPTER 6: CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The project titled **"Sentiment Analysis on Movie Reviews Using Machine Learning and NLP"** successfully demonstrates the application of Natural Language Processing and supervised machine learning techniques for automated sentiment classification.

The MovieMind system was developed to classify movie reviews into positive and negative sentiment categories. The implementation includes dataset collection, text preprocessing, feature extraction using TF-IDF, model training, performance evaluation, and Flask web application development.

Three machine learning algorithms were implemented and compared: Multinomial Naive Bayes, Linear Support Vector Machine, and Logistic Regression.

The experimental results demonstrate that Linear SVM achieved the highest performance, with an accuracy of **85.25%**, precision of **84.06%**, recall of **87.00%**, and F1-score of **85.50%**.

The best-performing model was integrated into the MovieMind web application, enabling users to enter movie reviews and obtain sentiment predictions in real time. The application also provides an analytics dashboard containing model performance comparisons, confusion matrices, and prediction history.

Overall, the project demonstrates how machine learning and NLP can be combined to develop a practical text classification application. It also provides hands-on experience in data preprocessing, feature engineering, model evaluation, and web application integration.

6.2 Future Enhancements

Although the current system provides satisfactory classification performance, several improvements can be implemented in future versions.

6.2.1 Deep Learning Integration

Advanced deep learning models such as LSTM, BiLSTM, and Transformer-based architectures can be implemented to capture more complex relationships in textual data.

6.2.2 BERT-Based Sentiment Analysis

Pre-trained language models such as BERT can be fine-tuned for movie review sentiment classification to improve contextual understanding.

6.2.3 Multiclass Sentiment Classification

The current binary classification system can be extended to support additional categories such as neutral, mixed, and strongly positive or negative sentiments.

6.2.4 Multilingual Support

The system can be enhanced to process movie reviews written in multiple languages.

6.2.5 Larger Dataset Integration

Larger and more diverse movie review datasets can be incorporated to improve the model's ability to generalize to different writing styles and review sources.

6.2.6 Aspect-Based Sentiment Analysis

Future versions can identify sentiment associated with specific movie aspects such as acting, screenplay, music, direction, and cinematography.

6.2.7 Cloud Deployment

The MovieMind application can be deployed on cloud platforms to make it accessible to users over the internet.

6.2.8 Advanced Dashboard Features

The dashboard can be enhanced with additional visualizations, review filtering, sentiment trends, and downloadable analytics reports.

6.3 Final Outcome

The final outcome of the project is a working machine-learning-based sentiment analysis application that:

- Processes English-language movie reviews.
- Classifies reviews into positive and negative categories.
- Compares three machine learning algorithms.
- Uses Linear SVM as the selected prediction model.
- Provides a Flask-based user interface.
- Displays model performance through an analytics dashboard.
- Maintains prediction history.

The project achieves its primary objective of developing an end-to-end movie review sentiment classification system using NLP and machine learning.

REFERENCES

1. Pang, B., Lee, L., and Vaithyanathan, S. (2002). “Thumbs up? Sentiment Classification Using Machine Learning Techniques.” *Proceedings of the ACL Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 79–86.

2. Pang, B., and Lee, L. (2004). “A Sentimental Education: Sentiment Analysis Using Subjectivity Summarization Based on Minimum Cuts.” *Proceedings of the 42nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 271–278.

3. Joachims, T. (1998). “Text Categorization with Support Vector Machines: Learning with Many Relevant Features.” *Proceedings of the European Conference on Machine Learning (ECML)*, pp. 137–142.

4. Salton, G., and Buckley, C. (1988). “Term-Weighting Approaches in Automatic Text Retrieval.” *Information Processing & Management*, 24(5), pp. 513–523.

5. Bird, S., Klein, E., and Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.

6. NLTK Project. “NLTK Documentation – Movie Reviews Corpus.” Available at: https://www.nltk.org/howto/corpus.html

7. Cornell University. “Movie Review Data.” Available at: https://www.cs.cornell.edu/people/pabo/movie-review-data/

8. Scikit-learn Developers. “Scikit-learn: Machine Learning in Python.” Available at: https://scikit-learn.org/

9. NLTK Project. “Natural Language Toolkit Documentation.” Available at: https://www.nltk.org/

10. Flask Documentation. “Flask Web Development Framework.” Available at: https://flask.palletsprojects.com/

11. Pedregosa, F., et al. (2011). “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, 12, pp. 2825–2830.

12. Salton, G., and McGill, M. J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.

APPENDIX A: PROJECT FILES

The project source code is organized into the following files:

File Name	Description
`collect_dataset.py`	Collects movie reviews
`explore_dataset.py`	Performs dataset exploration
`preprocessing.py`	Preprocesses review text
`feature_extraction.py`	Performs TF-IDF feature extraction
`train_model.py`	Trains and evaluates models
`test_predictions.py`	Tests sample predictions

File Name	Description
app.py	Runs the Flask application
index.html	MovieMind home page
dashboard.html	Analytics dashboard
style.css	Application styling
requirements.txt	Python dependencies

APPENDIX B: GENERATED OUTPUTS

The following outputs are generated during project execution:

1. Original movie review dataset.
2. Preprocessed movie review dataset.
3. Trained machine learning models.
4. Saved TF-IDF vectorizer.
5. Model comparison results.
6. Confusion matrix images.
7. Accuracy comparison chart.
8. Model performance comparison chart.
9. Prediction history CSV file.
10. MovieMind web application and dashboard.

END OF PROJECT REPORT